$$\begin{aligned} \mathbf{i}sz\Downarrow(v) \text{ or } ty*\Downarrow(v) &= \lambda i.\ \text{getbit}\ v\ i \\ \langle sz \times ty\rangle\Downarrow(v) &= ty\Downarrow(v[0]) +\!\!+ \ldots \\ &\quad +\!\!+ ty\Downarrow(v[sz-1]) \end{aligned}$$

(a) Converting a value to a bit vector

$$\mathbf{i}sz\Uparrow(b) = \begin{cases} n & \text{if } \forall_{0\le i<sz}\, b[i] \neq \textbf{poison} \\ & \text{such that } \forall_{0\le i<sz}\, b[i] = n.i \\ \textbf{poison} & \text{otherwise} \end{cases}$$

(b) Converting a bit vector to an integer

$$\frac{(\iota = \text{“}r = \textbf{load}\ ty, ty*\ op,\ \textbf{align}\ a\text{”}) \quad \text{Load}(M, [\![op]\!]_R, \text{bitwidth}(ty), a) \text{ fails}}{R, M \overset{\iota}{\hookrightarrow} \text{UB}}$$

$$\frac{(\iota = \text{“}\textbf{store}\ ty\ op_1, ty*\ op,\ \textbf{align}\ a\text{”}) \quad \text{Store}(M, [\![op]\!]_R, ty\Downarrow([\![op_1]\!]_R), a) \text{ fails}}{R, M \overset{\iota}{\hookrightarrow} \text{UB}}$$

$$\frac{\text{Load}(M, [\![op]\!]_R, \text{bitwidth}(ty)) = v}{R, M \overset{\iota}{\hookrightarrow} R[r \mapsto ty\Uparrow(v)], M}$$

$$\frac{\text{Store}(M, [\![op]\!]_R, ty\Downarrow([\![op_1]\!]_R)) = M'}{R, M \overset{\iota}{\hookrightarrow} R, M'}$$

Fig. 6. Semantics of **load** and **store**

of AddrBit denoting the ith bit of a non-poison pointer p. For vector types, $ty\Downarrow$ transforms values element-wise, where $+\!\!+$ denotes the bitvector concatenation.

$\mathbf{i}sz\Uparrow(b)$ transforms a bitwise value b to an integer of type $\mathbf{i}sz$ (Fig. 6b). Notation $n.i$ is used to represent the ith bit of a non-**poison** integer n. Type punning from pointer to integer yields **poison**. This is needed to justify redundant load-store pair elimination.[4] If any bit of b is **poison**, the result of $\mathbf{i}sz\Uparrow(b)$ is **poison**. For vector types, $ty\Uparrow$ transforms bitwise representations element-wise.

$ty*\Uparrow(b)$ transforms a bitvector b into a pointer of type $ty*$. If b is exactly all the bits of a pointer p in the right order, it returns p. Otherwise, it returns **poison**.

Now we define semantics of **load**/**store** operations. Function $\text{Load}(M, p, sz, a)$ returns the bits that correspond to pointer p if $\text{deref}_M(p, sz)$ and if p is a-aligned (i.e., $p \,\%\, a = 0$). **load** yields v if $\text{Load}(M, p, sz, a)$ returns a value v, or UB otherwise. The store operation $\text{Store}(M, p, b, a)$ stores the bit representation b into the memory M and returns the updated memory M' if p is dereferenceable and a-aligned. **store** is UB if $\text{Store}(M, p, b, a)$ fails, and updates the memory to M' otherwise.

4.10 Justifying Transformations in Practice

We now illustrate how our semantics can be used in practice in a compiler, and how one can informally reason about the correctness of optimizations. For example, we would like to justify that the following transformation performed by GVN is correct:

```
char *p = malloc(4);                char *p = malloc(4);
int v = (int)p;                     int v = (int)p;
if (v == 10)                  ⇒     if (v == 10)
    *(int*)v = 0;                       *(int*)10 = 0;
```

Under our semantics, this transformation is legal because integers do not track provenance. Moreover, the address of object `p` is observed by the comparison, and therefore the store through `(int*)10` is guaranteed to write to object `p`.

A useful way to think about this is that our semantics effectively take *control-flow* dependencies into account when determining which objects have had their addresses observed. In our example, the address of `p` is observed within the then block since it was used in a comparison along the control-flow that leads to the block. Hence, we allow `(int*)10` to refer to object `p`. This sort of reasoning can be directly implemented in a compiler.

[4]Redundant load-store pair elimination means removing ‘`v = load i64 ptr; store v, ptr`’. If reading a logical pointer as integer implicitly casts the pointer, removing this load-store pair would eliminate a cast and hence be illegal. This is discussed further in Section 8.